Exercise Week 5: Toolformer.

Task 1: Dataset Creation (7pts)

First off: Read the Toolformer Paper carefully. To set up the project initialize the code environment on the cluster by cloning the provided repository. Use an instance with either an A40 or A100 GPU (A40 is sufficient, A100 quite a bit faster).

> **Hint**
>
> A possible command for `salloc` should contain: `--gpus=A40:1`.
> You can work in the `-p gpu-interactive` partition.

Then set up the dependencies via `uv sync`. **It is important to do this step on a node with access to a GPU**. You download the dataset by running `uv run scripts/download_dataset.py`. This will download the $\approx 700MB$ Alpaca-Cleaned dataset into `data`.

The general structure of the project is as follows:

```
<REPO>
├─data //the directory for storing and accessing data
├─src //All source files, containing reusable functions
├─scripts // One-off scripts, i.e. they run, they do something complex, but their parts do not get reused
elsewhere
├─cfg.toml // contains settings that might be relevant for you
├─project_paths.py // contains references to all these folders, the config, and more
├─pyproject.toml // the dependencies
```

Take note of the images in Reference Information, to help you understand the overarching approach.

Have a careful look at the configuration in `cfg.toml`. Some of the keys in there will be referenced later and in the code.

Step 1: API Call Mining

Within the group folder (`/sc/projects/sci-lippert/intelligent-agents/model_checkpoints/`) we provide you with the checkpoints to some bigger models, which we tested to run on both a single A40 or a single A100 GPU.

The provided models are:

- `Qwen/Qwen3-14B`
- `Qwen/Qwen2.5-14B-Instruct`
- `microsoft/phi-4`

We would recommend using Qwen3 or Phi-4 for this task, but feel free to experiment with (the) other models.

For this task you are to modify `scripts/generate_dataset.py`. Follow the instructions in the file carefully.

Additionally you will need to come up with a prompt for each of the available tools: `[qa,runcode,wikisearch,calculate,calendar]`, and put those in `data/prompts`.

> **Expected Result**
>
> After this your `data` folder should contain a `generated_dataset.json`. This dataset should contain instruction tuning triplets to be used in the next steps. The columns are "instruction" and "input" from alpaca-clean, as well as "output" containing the "enhanced" output.
> Your `data/prompts` folder should contain a file for each available tool: We give an example in `cal_prompt.txt`, for the calendar tool.

> **Hint**
>
> When using Qwen3, make sure to append the `prompt_postfix` from the `cfg.toml` to all your prompts, to prevent the model from entering a thinking mode (or alternatively filter the `<think> ... </think>` section)

Step 2: Running the Tools.

Next we will have to run the tool-calls and add in the results to the generated data. For this follow the instructions in `scripts/run_tools.py`.

> **Expected Result**

After this your `data` folder should contain a `generated_data_after_run.json`. This dataset should contain the following columns: "instruction" and"input" form the original dataset, as well as "output". Where "output" contains the resulting extended output after running the tool and adding it's output.

Step 3: Filtering the Useful API Calls.

Filter the generated examples as described in the section "Filtering API Calls" in the paper. For this you are to modify `filter_dataset.py`. Follow the instructions in the corresponding file carefully. Additionally have a look at HuggingFace Perplexity, to get an understanding of the measure we will be calculating. In `filter_dataset.py` you will have

Expected Result

After this you should have a filtered `finetune_dataset.json` file in the `data` folder. This dataset should contain instruction tuning pairs to be used with your chosen model, which we will finetune in the next step. The dataset should have the columns "instruction" and "input" from the original data, as well as "output" with the filtered output.

Task 2: Finetuning a Model (3pts)

Take one of the smaller litgpt supported models, and apply the finetuning on your newly created dataset.

We recommend fine-tuning `TinyLlama/TinyLlama-1.1B-Chat-v1.0`. For the finetuning you will have to make a modified model first by running `scripts/make_modified_model.py`. This model can then be used with your `finetune_dataset.json`, generated in the previous step.

To make the Finetuning work you will have to complete the `compute_loss` function in `src/train.py`.

Follow the instructions in `src/train.py` to complete the task.

Hint

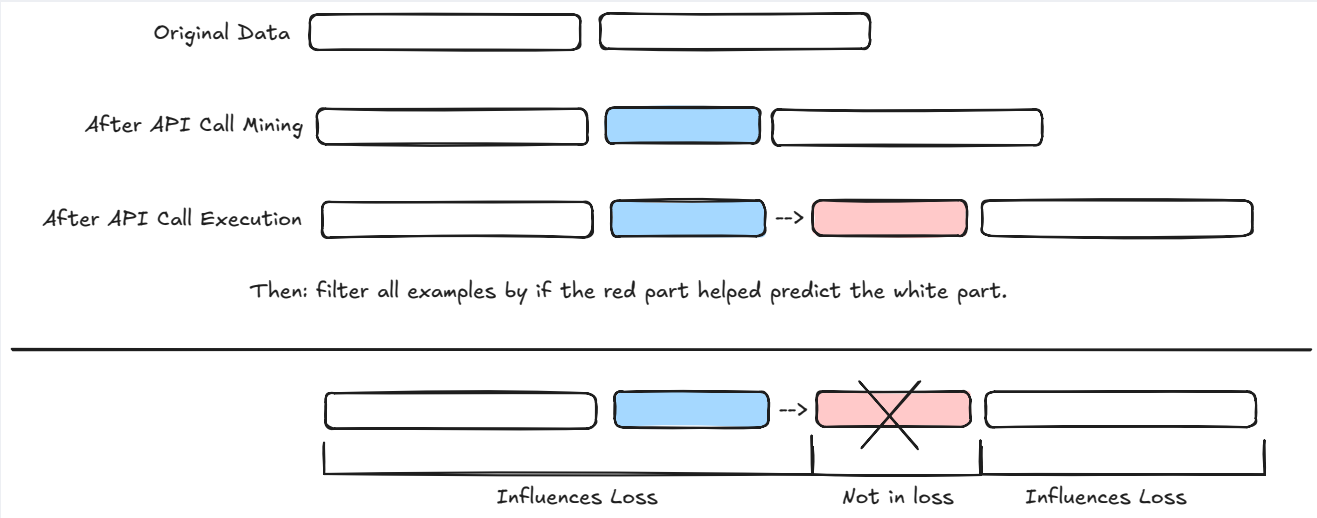
The loss should be calculated "normal" as if you are doing next-token prediction but you need to set the token-ids in the labels array to be ignored in the CrossEntropyLoss.

Good Luck! I hope you can take something useful from the exercise.

> If you have done everything you should now be in possession of a working implementation for Toolformer. ~Noel

⚠ If there are any unexpected issues, let us know in the QA forum.

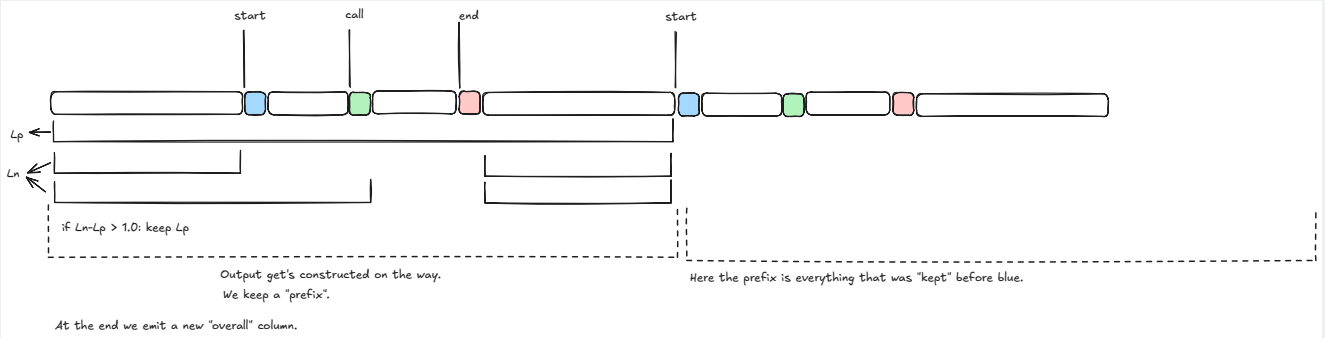
Reference Information



In the exercise I drew the above picture:

1. The original Data is comprised of model outputs, that have some index i where we can split the string to insert the API calls.
2. After inserting at i we now have the extended data, and by virtue of controlling the exact string used to demark this area (`◁tool_start▷`, `◁tool_end▷` or similar token) we can easily extract these API calls.
3. We then execute them, inserting the "execute toolcall token `◁⟶▷` and the result **before** the `◁tool_end▷` token.
4. Afterwards we have to filter the examples, by checking if the perplexity for the ending white part is lowered by the presence of our API call result. *Note: it specifically is about the influence of the red part, not the overall added tool call.*

5. Lastly we fine-tune the model on the remaining examples but ignore the red part in our loss calculation.



Here is an additional drawing for how the filtering can be done in steps, if you have more than one API call in your generated dataset.